SENA — Servicio Nacional de Aprendizaje

Tecnólogo en Análisis y Desarrollo de Software

Ficha de formación: 3145556

Centro de formación: Neiva, Huila

Manual de Despliegue — School Guardian (Guardian Escolar)

Guardián Escolar

Integrantes del proyecto:

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Neiva, 6 de octubre de 2026

Tabla de contenido

1. Información General y Control de Versiones....4
1.1 Ficha Técnica del Despliegue....4
1.1.1 Identificación del sistema y versión (Release Tag)....4
1.1.2 Fecha, hora y ventana operativa programada....4
1.1.3 Entorno de destino (VPS / Staging / Producción)....5
1.1.4 Matriz de roles y responsables (DevOps, Tech Lead, QA, DBA)....5
1.2 Historial de Versiones del Documento....5
1.3 Resumen de Cambios (Release Notes)....6
1.3.1 Nuevas funcionalidades....6
1.3.2 Correcciones de errores (Bug fixes)....6
1.3.3 Cambios de infraestructura y dependencias....7
1.3.4 Cambios incompatibles (Breaking changes)....7
2. Alcance, Topología y Arquitectura....8
2.1 Componentes y Artefactos en Alcance....8
2.1.1 Panel de administración (Portainer CE)....8
2.1.2 Capa Frontend Web (Angular SPA / Servidor Web)....9
2.1.3 Capa Backend API (Java Spring Boot / C# .NET Core)....9
2.1.4 Capa de Persistencia (SQL Server en RDS)....10
2.2 Diagrama de Arquitectura de Red e Infraestructura....10
2.2.1 Topología del servidor (EC2)....10
2.2.2 Red interna de contenedores (Docker Bridge Network)....11
2.2.3 Enrutamiento y Reverse Proxy con certificados TLS/HTTPS....11
2.3 Matriz de Ambientes, Direccionamiento IP y Puertos....12
3. Requisitos Previos y Preparación (Pre-deployment)....12
3.1 Checklist de Prerrequisitos del Sistema Operativo y EC2....12
3.1.1 Recursos de hardware disponibles (CPU, RAM, disco)....12
3.1.2 Estado del motor Docker y socket local....13
3.2 Estrategia y Ejecución de Respaldos (Backups)....13
3.2.1 Respaldo lógico de base de datos (Dump / Snapshot)....13
3.2.2 Respaldo de volúmenes persistentes de Docker....14
3.2.3 Respaldo de configuraciones de Stacks y variables anteriores....14
3.3 Gestión de Variables de Entorno y Secretos....14
3.3.1 Parámetros de conexión a base de datos....14
3.3.2 Claves de firmado de tokens (JWT Secret Keys)....15
3.3.3 Límites de memoria y runtime....15
3.4 Protocolo de Comunicación y Ventana de Mantenimiento....15
4. Procedimiento de Despliegue Paso a Paso (Runbook)....16
4.1 Verificación y Acceso a la Plataforma Base (Portainer CE)....16
4.2 Despliegue y Migración de Base de Datos....17
4.2.1 Ejecución de scripts DDL/DML o migraciones (Liquibase)....17
4.2.2 Verificación de integridad de esquemas, índices y usuarios....18

4.3 Despliegue y Orquestación del Stack de Aplicación....18
4.3.1 Carga del archivo docker-compose.yml....18
4.3.2 Inyección de variables de entorno del Stack....19
4.3.3 Despliegue de servicios y actualización de imágenes de contenedor....19
4.4 Configuración de Enrutamiento, Dominio y Certificados SSL/TLS....19
5. Plan de Verificación, Pruebas y Evidencias (Smoke Tests)....20
5.1 Matriz de Enlaces y Endpoints de Verificación de Servicios....20
5.1.1 Acceso al panel administrativo (Portainer)....20
5.1.2 Interfaz web pública (Angular / Vercel)....21
5.1.3 Endpoints de salud del Backend....21
5.1.4 Documentación interactiva de la API (Scalar)....21
5.1.5 Conectividad TCP directa a motor de base de datos....22
5.2 Pruebas de Humo Funcionales (Smoke Tests)....22
5.2.1 Flujo de autenticación y emisión de tokens....22
5.2.2 Pruebas de lectura y consulta de datos....22
5.2.3 Pruebas de escritura y persistencia transaccional....23
5.3 Inspección de Logs y Monitoreo de Recursos....23
5.3.1 Revisión de trazas de inicio sin excepciones críticas....23
5.3.2 Monitoreo de consumo de CPU y memoria RAM en contenedores....23
5.4 Registro de Evidencias Visuales (Screenshots)....23
5.4.1 Captura del panel de Portainer con contenedores en estado Running....24
5.4.2 Captura de visor de logs de los servicios Backend....24
5.4.3 Captura de respuesta HTTP 200 de los Health Checks....25
5.4.4 Captura de interfaz web React/Angular con consola DevTools limpia....26
5.4.5 Captura de registro de migración exitosa en base de datos....26
6. Anexos....27
6.1 Datos de acceso y direcciones....27
6.2 Cuentas de prueba (entorno de testing)....28
6.3 Acceso SSH....28
6.4 Servicio GPS (ms-gps) — configuración del tracker VT03F....28
6.5 Frontend Web (Vercel) — variables de entorno....29
6.6 App Móvil (React Native + Expo) — variables y compilación....29
6.7 Troubleshooting....30
6.8 Respaldos relevantes....30

Documento operativo de despliegue de la plataforma **School Guardian** sobre AWS. Las evidencias visuales del despliegue están incorporadas en la sección §5.4.

---

1. Información General y Control de Versiones

1.1 Ficha Técnica del Despliegue

1.1.1 Identificación del sistema y versión (Release Tag)

Campo	Valor
Nombre del sistema	School Guardian (Guardian Escolar)
Release Tag / Versión	`1.0.0`
Fecha de build	`2026-10-16`
Repositorio Backend	`school-guardian/backend` (rama `develop`)
Repositorio Base de Datos	`school-guardian/database` (Liquibase)
Frontend Web	`scholl-guardian-project/dvlp-front/dvlp-web` (Angular)
Aplicación Móvil	`scholl-guardian-project/dvlp-front/dvlp-movil/guardian-escolar` (React Native / Expo)
Autor del despliegue	Juan Chala

1.1.2 Fecha, hora y ventana operativa programada

Campo	Valor
Fecha de despliegue	`2026-10-15`
Hora de inicio	`20:00`
Hora estimada de fin	`23:00`
Ventana de mantenimiento	`20:00–23:00`

Campo	Valor
Duración total	`3 horas`

1.1.3 Entorno de destino (VPS / Staging / Producción)

Campo	Valor
Tipo de entorno	AWS EC2 (equivalente a VPS) — entorno de **pruebas / uso intermitente** (no productivo)
Región AWS	`us-east-2` (Ohio), zona `us-east-2a`
ID de instancia	`i-0911016bb4a900105`
Sistema Operativo	Debian GNU/Linux 13 (trixie), kernel 6.12
IP pública	**`<IP>`** (Elastic IP fija)
Nombre público (DNS)	`<HOST>` (sin comprar dominio)

1.1.4 Matriz de roles y responsables (DevOps, Tech Lead, QA, DBA)

Rol	Nombre	Responsabilidad
DevOps / Infraestructura	Juan Chala	EC2, Docker, Caddy, Kong, Portainer, Security Group
Backend Developer	Sharik Dayanna Rojas Ibarra, Johan Smith Santamaria Fernández, Juan Pablo Chala Ramírez	Microservicios .NET / Spring Boot
Frontend / Móvil	Sharik Dayanna Rojas Ibarra, Johan Smith Santamaria Fernández, Juan Pablo Chala Ramírez	Angular (Vercel) y React Native (EAS)

1.2 Historial de Versiones del Documento

Versión	Fecha	Autor	Descripción del cambio
1.0	2026-10-15	Juan Chala	Versión inicial del manual de despliegue
1.1	2026-10-16	Juan Chala	Reestructuración al formato de Plan de Despliegue; migración a

Versión	Fecha	Autor	Descripción del cambio
			Elastic IP `<IP>`; integración de Portainer CE
1.2	2026-10-18	Juan Chala	Flujo de Portainer corregido: setup token generado por Portainer (sin credenciales ni token hardcodeados); evidencias visuales §5.4

1.3 Resumen de Cambios (Release Notes)

1.3.1 Nuevas funcionalidades

- **Sistema GPS completo** (`ms-gps`): servidor TCP en el puerto `8842` (protocolo VT03F) + API REST vía Kong (`/api/gps`). Persistencia en memoria (no requiere base de datos).
- **Panel de administración Portainer CE**: gestión visual de contenedores en `https://portainer.<HOST>`.
- **Servicio de información olvidada** (`ms-forgot-information`).
- **Notificaciones** (`ms-notification`) integradas con el backend vía Kafka.

1.3.2 Correcciones de errores (Bug fixes)

- **Web en blanco / datos no cargaban**: corregida la ruta de Kong para `/user/api/*` (mapeaba mal hacia `ms-user-management`).
- **Error 500 en `/schools` y `/schools/{id}/campuses`**: por tipo `School.Phone` (`INT` → `BIGINT`) y columnas `Latitude`/`Longitude` faltantes.

- **Columnas faltantes en `UserManagement.Person`** (`CityId`, `SchoolId`).
- **`ms-iam` caído tras reinicio** por fallo de conexión a RDS en el arranque (reinicio del contenedor resuelve).

1.3.3 Cambios de infraestructura y dependencias

- **IP pública pasó de dinámica a Elastic IP fija**: `<IP>`. Ya no cambia al reiniciar la instancia.
- **Dominio y TLS** regenerados para `<HOST>` (Let's Encrypt vía Caddy).
- **Integración de Portainer CE** (nuevo compose `docker-compose.portainer.yml`) detrás de Caddy, sin abrir puertos nuevos.
- Base de datos migrada a **AWS RDS SQL Server Express** (free tier), separando la persistencia del servidor de aplicación.

1.3.4 Cambios incompatibles (Breaking changes)

- **Cambio de IP pública**: todas las variables que apuntaban a la IP anterior (`<IP>`, `<IP>`) deben actualizarse a `<IP>` (Vercel, EAS/móvil, tracker GPS).
- **Acceso público a Portainer** por nuevo subdominio; requiere agregar `--trusted-origins` (los navegadores con versiones nuevas de Portainer 2.27.7+ rechazan login sin este flag).
- **`iam` changeset 019**: su aplicación modifica datos existentes; se marcó como `EXECUTED` manualmente tras insertar las filas requeridas (ver §4.2).

2. Alcance, Topología y Arquitectura

2.1 Componentes y Artefactos en Alcance

#	Componente	Tecnología	Artefacto	Ubicación
1	Panel de administración	Portainer CE 2.45 (LTS)	`docker-compose.portainer.yml`	EC2 → `portainer:9000`
2	Reverse proxy / TLS	Caddy 2	`deploy/Caddyfile.gateway`	EC2 → `caddy` (80/443)
3	API Gateway	Kong (DB-less)	`ms-api-gateway/kong/kong.yml`	EC2 → `kong:8000`
4	Backend API	C# .NET + Java Spring Boot	`docker-compose.yml`	EC2 → `ms-*`
5	Persistencia	SQL Server (RDS)	`database/*` (Liquibase)	AWS RDS `:1433`
6	Frontend Web	Angular SPA	`dvlp-web`	Vercel
7	App Móvil	React Native / Expo	`guardian-escolar`	EAS Build (APK/IPA)

2.1.1 Panel de administración (Portainer CE)

- **Imagen:** `portainer/portainer-ce:lts` (2.45.2).
- **Despliegue:** compose propio `docker-compose.portainer.yml`, unido a la red `sg-services-network`.
- **Publicación: ningún puerto al host**. Caddy hace `reverse_proxy` hacia `portainer:9000`.
- **Acceso:** `https://portainer.<HOST>`.

- **Gestión:** monta `<RUTA_LOCAL> para administrar los contenedores del host.
- **Flag requerido:** `--trusted-origins https://portainer.<HOST>` (evita el error *Origin invalid* detrás de proxy).
- **Setup token (primer acceso): no se fija en el compose** (nada hardcodeado). Portainer **genera el token solo** al arrancar y lo imprime en los logs; el procedimiento para obtenerlo está en §4.1.

2.1.2 Capa Frontend Web (Angular SPA / Servidor Web)

- **Framework:** Angular (build SPA) — la plantilla original menciona *Servidor Web Nginx*, pero en este despliegue el frontend se sirve como **SPA estática en Vercel** (CDN), no con Nginx propio.
- **Build:** `npx ng build --configuration spa` → `dist/school-guardian/browser`.
- **Hosting:** Vercel (`https://guardian-escolar.vercel.app`), rewrites SPA a `index.html`.
- **Variables de build (Vercel):** `API_URL`, `API_FORGOT_INFORMATION_URL`.

2.1.3 Capa Backend API (Java Spring Boot / C# .NET Core)

Microservicio	Tecnología	Puerto interno
`ms-iam`	.NET	8080
`ms-user-management`	.NET	8080
`ms-school-management`	.NET	8080
`ms-fleet`	.NET	8080
`ms-route`	.NET	8080
`ms-notification`	.NET	8080
`ms-forgot-information`	.NET	8080

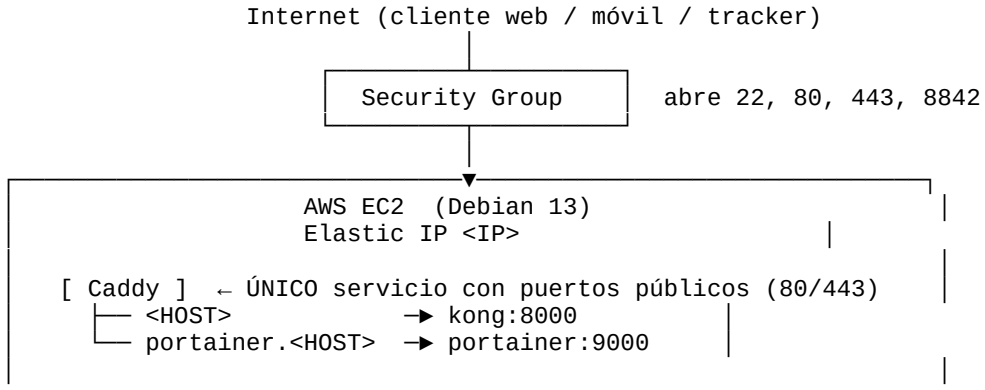
Microservicio	Tecnología	Puerto interno
`ms-gps`	**Java Spring Boot**	8080 (REST) / **8842** (TCP)
`kafka`	Apache Kafka (KRaft)	9092

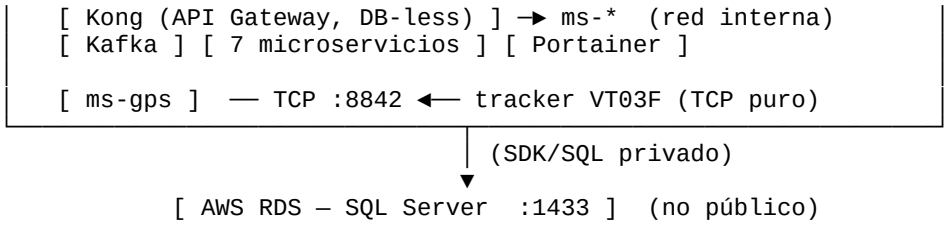
2.1.4 Capa de Persistencia (SQL Server en RDS)

- **Motor:** Microsoft SQL Server (Express) en **AWS RDS**.
- **Host:** `<HOST_BASE_DATOS>:1433`.
- **Base de datos:** `school-guardian`; usuario `<USUARIO>`.
- **Esquemas:** `Iam`, `UserManagement`, `School`, `Fleet`, `Route`, `Notification`, `Gps`, `Geographic`.
- **Migraciones: Liquibase** (la plantilla menciona Flyway/EF Core; en este proyecto se usa Liquibase 4.29 sobre el repo `database/`).
- **RDS no es accesible desde internet**: solo desde la instancia EC2 (Security Group).

2.2 Diagrama de Arquitectura de Red e Infraestructura

2.2.1 Topología del servidor (EC2)





2.2.2 Red interna de contenedores (Docker Bridge Network)

- Red externa compartida: **`sg-services-network`** (bridge).
- Regla de oro: **ningún microservicio publica puertos al host**; se comunican por nombre de contenedor (`http://ms-iam:8080`, `kafka:9092`, `portainer:9000`, `kong:8000`).
- Caddy y Portainer se conectan a la misma red para el `reverse_proxy`.

2.2.3 Enrutamiento y Reverse Proxy con certificados TLS/HTTPS

- **Caddy** es la única puerta HTTPS pública (443/80). Obtiene y renueva certificados **Let's Encrypt** automáticamente (challenge HTTP-01) para los dominios `sslip.io`.
- Caddy hace `reverse_proxy` a:
  - `kong:8000` (toda la API), y
  - `portainer:9000` (panel).
- **Kong** aplica ruteo por path y valida JWT (plugin `sg-jwt`):

Ruta pública (Kong)	**Servicio destino**
`/api/v1/auth`, `/api/v1/password`	`ms-iam`
`/user` (→ `/user/api`)	`ms-user-management`
`/fleet`	`ms-fleet`
`/school-management`	`ms-school-management`
`/notification`	`ms-notification`
`/route`	`ms-route`

Ruta pública (Kong)	Servicio destino
`/api/gps`	`ms-gps` (público, sin JWT)

2.3 Matriz de Ambientes, Direccionamiento IP y Puertos

Componente	Host / IP	Puerto	Protocolo	¿Público?
Caddy (HTTPS)	`<IP>` / `<HOST>`	443	HTTPS	**Sí**
Caddy (HTTP → redirect)	`<IP>`	80	HTTP	**Sí** (redirige a 443)
SSH (administración)	`<IP>`	22	SSH	Sí (restringido)
GPS TCP (tracker VT03F)	`<IP>`	8842	TCP puro	**Sí**
Portainer	vía Caddy	9000	HTTP (interno)	No (proxy)
Kong	`kong` (Docker)	8000 / 8001	HTTP	No
Microservicios `ms-*`	names Docker	8080	HTTP	No
Kafka	`kafka`	9092	TCP	No
RDS SQL Server	`<HOST_BASE_DATOS>`	1433	TDS	No (solo EC2)

---

3. Requisitos Previos y Preparación (Pre-deployment)

3.1 Checklist de Prerrequisitos del Sistema Operativo y EC2

3.1.1 Recursos de hardware disponibles (CPU, RAM, disco)

Recurso	Valor real
vCPU	2
Memoria RAM	~1.9 GiB (uso en reposo ~1.5 GiB)
Swap	4 GiB (configurado para builds/imágenes)

Recurso	Valor real
Disco raíz	7.7 GiB (uso ~22 %)
Sistema Operativo	Debian 13 (trixie)

Comandos de verificación:

```
nproc
free -h
df -h /
```

3.1.2 Estado del motor Docker y socket local

Componente	Versión
Docker Engine	29.9.1
Docker Compose	v5.6.0

Verificación:

```
docker --version
docker compose version
docker ps                        # contenedores en ejecución
ls -l <RUTA_LOCAL>      # socket requerido por Portainer
```

3.2 Estrategia y Ejecución de Respaldos (Backups)

3.2.1 Respaldo lógico de base de datos (Dump / Snapshot)

- **Snapshot manual de RDS** (recomendado) antes de aplicar migraciones:
  - Consola AWS → RDS → Base de datos → *Actions* → *Take snapshot*.
- **Respaldo lógico** vía `sqlcmd` (equivalente a dump):

```
docker run --rm --entrypoint <RUTA_LOCAL> \
  mcr.microsoft.com/mssql-tools:latest \
  -S <HOST_BASE_DATOS> \
  -U <USUARIO> -P "$MSSQL_SA_PASSWORD" -C \
  -Q "BACKUP DATABASE [school-guardian] TO DISK = N'<RUTA_LOCAL>
```

3.2.2 Respaldo de volúmenes persistentes de Docker

Volúmenes relevantes: `caddy-data`, `caddy-config`, `portainer-data`, `kafka-data`.

```
docker run --rm -v portainer-data:/data -v "$PWD":/backup alpine \
  tar czf /backup/portainer-data.tgz -C /data .
```

3.2.3 Respaldo de configuraciones de Stacks y variables anteriores

Antes de cada cambio de configuración se crean copias con sufijo `.bak-*`:

Archivo	**Respaldo**
`ms-api-gateway/kong/kong.yml`	`kong.yml.bak-pre-user-v1`, `kong.yml.bak-pre-notif`, `kong.yml.bak-pre-ip`, `kong.yml.bak-pre-eip`
`deploy/Caddyfile.gateway`	`Caddyfile.gateway.bak-pre-ip`, `Caddyfile.gateway.bak-pre-eip`, `Caddyfile.gateway.bak-pre-portainer`

3.3 Gestión de Variables de Entorno y Secretos

3.3.1 Parámetros de conexión a base de datos

Cada microservicio tiene su `.env` (no versionado) con la conexión a RDS:

```
ConnectionStrings__DefaultConnection=Server=<HOST_BASE_DATOS>;Database=school-
guardian;User Id=<USUARIO>;Password=<REDACTADO>;TrustServerCertificate=True;
```

3.3.2 Claves de firmado de tokens (JWT Secret Keys)

- `ms-iam` firma los JWT (clave RSA); Kong valida con la clave pública (`sg-jwt`).
- Las claves/claves privadas viven en los `.env` de `ms-iam` y en la configuración de Kong (no se versionan).

3.3.3 Límites de memoria y runtime

- Servicios .NET: `ASPNETCORE_ENVIRONMENT=Production`.
- Servicios Java: parámetros JVM `-Xmx` acotados (instancia pequeña con swap).

3.4 Protocolo de Comunicación y Ventana de Mantenimiento

- **Comunicación:** todo cambio se avisa al Tech Lead y QA antes de la ventana.
- **Ventana:** 2026-10-15, de 20:00 a 23:00.
- **Rollback:** restaurar backups `.bak-*` de Kong/Caddy y reiniciar contenedores; en BD, restaurar snapshot de RDS.

---

4. Procedimiento de Despliegue Paso a Paso (Runbook)

4.1 Verificación y Acceso a la Plataforma Base (Portainer CE)

1. Confirmar que Portainer está arriba:

```
docker ps --format '{{.Names}}\t{{.Status}}' | grep portainer
# portainer   Up ...
```

2. Levantar Portainer si no está:

```
cd <RUTA_LOCAL>
docker compose -f docker-compose.portainer.yml up -d
```

3. Acceder por navegador a:

```
https://portainer.<HOST>
```

4. En el **primer acceso**, Portainer pide un **setup token**. Obtenerlo leyendo los logs del contenedor (Portainer **lo genera solo**; no está en ningún archivo ni en el compose):

```
ssh -i <RUTA_LOCAL> admin@<IP> \
  'docker logs portainer 2>&1 | grep setup_token | tail -1'
# setup_token=<REDACTADO>
```

5. Pegar el token en la pantalla y **crear el usuario administrador propio** (usuario y contraseña de **mínimo 12 caracteres** cada uno) → *Create user* → **Local** → **Connect**.

Flujo del setup token. Es un mecanismo de seguridad de Portainer para impedir que cualquiera cree el primer administrador. Solo existe **mientras no haya ningún admin creado**; una vez creado, la pantalla pasa a ser un login normal y el token desaparece. Se imprime en `docker logs portainer` en **cada arranque**, por lo que **cambia al reiniciar** el contenedor y hay una ventana de ~5 minutos para usarlo (si expira o se reinicia, basta con volver a leerlo de los logs). **No se hardcodea** en el compose: este solo define `--trusted-origins`.

4.2 Despliegue y Migración de Base de Datos

4.2.1 Ejecución de scripts DDL/DML o migraciones (Liquibase)

Las migraciones viven en el repo **`database/`** (Liquibase 4.29), con un módulo por servicio (`ms-iam-db`, `ms-user-management-db`, `ms-school-db`, `ms-fleet-db`, `ms-route-db`, `ms-notification-db`, `ms-geographic-db`, `ms-gps-db`).

- **Contra SQL Server local (dev):**

```
cd database
docker compose up liquibase-iam liquibase-school liquibase-fleet liquibase-gps
liquibase-geographic liquibase-notification liquibase-user-management
```

- **Contra RDS (despliegue):** ejecutar cada `liquibase-*` apuntando el `LIQUIBASE_COMMAND_URL` al host de RDS (`databaseName=school-guardian`).

En este despliegue las migraciones se aplicaron al **100 %** (8 esquemas, 133 changesets, 32 tablas). El **changeset `019`** de `ms-iam-db` (DML) se desbloqueó insertando manualmente las 73 filas requeridas y marcando el changeset como `EXECUTED` con el checksum de Liquibase.

4.2.2 Verificación de integridad de esquemas, índices y usuarios

Correcciones no destructivas aplicadas por diferencias entre el código y las migraciones:

```
-- UserManagement.Person
ALTER TABLE [UserManagement].[Person] ADD [CityId]   UNIQUEIDENTIFIER NULL;
ALTER TABLE [UserManagement].[Person] ADD [SchoolId] UNIQUEIDENTIFIER NULL;

-- School.School
ALTER TABLE [School].[School]       ADD [Latitude]  DECIMAL(18,6) NULL;
ALTER TABLE [School].[School]       ADD [Longitude] DECIMAL(18,6) NULL;

-- School.SchoolCampus
ALTER TABLE [School].[SchoolCampus] ADD [Latitude]  DECIMAL(18,6) NULL;
ALTER TABLE [School].[SchoolCampus] ADD [Longitude] DECIMAL(18,6) NULL;

-- Ajuste de tipo para compatibilidad con la entidad
ALTER TABLE [School].[School] ALTER COLUMN [Phone] BIGINT NOT NULL;
```

Verificación (ejemplos):

```
SELECT name FROM sys.schemas;                                   -- esquemas
SELECT COUNT(*) FROM Iam.Role;                                  -- datos base
SELECT COL_LENGTH('School.School','Latitude');                  -- columna existe
```

4.3 Despliegue y Orquestación del Stack de Aplicación

4.3.1 Carga del archivo `docker-compose.yml`

Tres stacks independientes en `<RUTA_LOCAL>

Archivo	Contenido
`docker-compose.yml`	Kafka + los 7 microservicios + Kong
`docker-compose.gateway.yml`	Caddy (80/443)
`docker-compose.portainer.yml`	Portainer CE

```
cd <RUTA_LOCAL>
docker network create sg-services-network 2>/dev/null || true
docker compose up -d                        # servicios + kong
docker compose -f docker-compose.gateway.yml   up -d   # caddy
docker compose -f docker-compose.portainer.yml up -d   # portainer
```

4.3.2 Inyección de variables de entorno del Stack

Cada servicio lee su `.env` (RDS, Kafka `kafka:9092`, JWT, etc.). Verificar que existan antes de levantar:

```
ls -1 */\.env
```

4.3.3 Despliegue de servicios y actualización de imágenes de contenedor

```
docker compose pull          # actualizar imágenes base
docker compose up -d --build # reconstruir y recrear
docker compose ps            # estado
```

4.4 Configuración de Enrutamiento, Dominio y Certificados SSL/TLS

1. Editar `deploy/Caddyfile.gateway` con los dos sitios:

```
<HOST> {
	encode gzip
	reverse_proxy kong:8000 {
		header_up Host {host}
```

```
                header_up X-Real-IP {remote_host}
        }
        log { output stdout }
}

portainer.<HOST> {
        encode gzip
        reverse_proxy portainer:9000 {
                header_up Host {host}
                header_up X-Real-IP {remote_host}
        }
        log { output stdout }
}
```

2. Reiniciar Caddy (obtiene/renueva certificados Let's Encrypt automáticamente):

```
docker restart caddy
docker logs --tail 40 caddy | grep -i "certificate obtained"
# certificate obtained successfully  identifier=<HOST>
# certificate obtained successfully  identifier=portainer.<HOST>
```

3. (Opcional) Ajustar orígenes CORS en `ms-api-gateway/kong/kong.yml` y recargar:

```
docker restart kong
```

---

5. Plan de Verificación, Pruebas y Evidencias (Smoke Tests)

5.1 Matriz de Enlaces y Endpoints de Verificación de Servicios

5.1.1 Acceso al panel administrativo (Portainer)

Recurso	**URL**	**Resultado esperado**
Portainer	`https://portainer.<HOST>`	HTTP 200 (pantalla de inicio de sesión)

5.1.2 Interfaz web pública (Angular / Vercel)

Recurso	**URL**	**Resultado esperado**
Web	`https://guardian-escolar.vercel.app`	Carga login sin errores en consola

5.1.3 Endpoints de salud del Backend

Los microservicios **no exponen** `/actuator/health` ni `/healthz` de forma pública (devuelven 404/403). La salud se verifica en la capa de contenedores y de gateway.

Verificación	**Comando**	**Esperado**
Estado de contenedores	`docker ps`	Todos `Up`; `kong → (healthy)`
Gateway API viva	`POST /api/v1/auth/login`	HTTP 200 (emite token)

5.1.4 Documentación interactiva de la API (Scalar)

La documentación interactiva **no se hace con Swagger UI**, sino con **Scalar**. Cada microservicio **.NET** expone Scalar en la ruta `/scalar/v1` (protegida por el JWT del gateway, Kong):

Servicio	**URL (vía Caddy)**
ms-route	`https://<HOST>/route/scalar/v1`
ms-fleet	`https://<HOST>/fleet/scalar/v1`
ms-school-management	`https://<HOST>/school-management/scalar/v1`
ms-notification	`https://<HOST>/notification/scalar/v1`

Servicio	URL (vía Caddy)
ms-user-management	`https://<HOST>/user/api/scalar/v1`

El esquema OpenAPI en JSON queda en la misma base (`/scalar/v1` para la UI y `/openapi/v1.json` para el JSON). El acceso requiere token válido (**HTTP 401** sin autenticar). `ms-gps` es Spring Boot y no expone Scalar.

5.1.5 Conectividad TCP directa a motor de base de datos

Desde la instancia EC2 (RDS es privado):

```
timeout 6 bash -c 'cat < /dev/null > /dev/tcp/<HOST_BASE_DATOS>/1433' \
  && echo "RDS 1433 ABIERTO" || echo "RDS 1433 NO alcanzable"
```

5.2 Pruebas de Humo Funcionales (Smoke Tests)

5.2.1 Flujo de autenticación y emisión de tokens

```
BASE=https://<HOST>
curl -s -m 20 -X POST -H 'Content-Type: application/json' \
  -d '{"email":"<CORREO_ADMIN>","password":"<CONTRASENA_ADMIN>"}' \
  "$BASE/api/v1/auth/login"
# → 200 con { "accessToken": "...", "refreshToken": "..." }
```

5.2.2 Pruebas de lectura y consulta de datos

Con el `accessToken`:

Endpoint	Esperado
`GET /api/v1/auth/profile`	200
`GET /user/api/students`	200
`GET /route/api/routes`	200
`GET /fleet/api/buses`	200
`GET /school-management/api/v1/schools`	200
`GET /api/gps/devices/{imei}/connected`	200

5.2.3 Pruebas de escritura y persistencia transaccional

- Crear/actualizar un recurso (p. ej. una ruta o un estudiante) y verificar 200/2xx + lectura posterior que confirme la persistencia en RDS.
- Caso de evidencia: `POST /route/api/routes` (crear ruta) → respuesta 2xx, verificada con `GET /route/api/routes/{id}` → **HTTP 200**.

5.3 Inspección de Logs y Monitoreo de Recursos

5.3.1 Revisión de trazas de inicio sin excepciones críticas

```
docker logs --tail 50 ms-iam      | grep -iE "error|exception"   # sin errores
críticos
docker logs --tail 50 ms-gps      | grep -i "GPS conectado"      # tracker
```

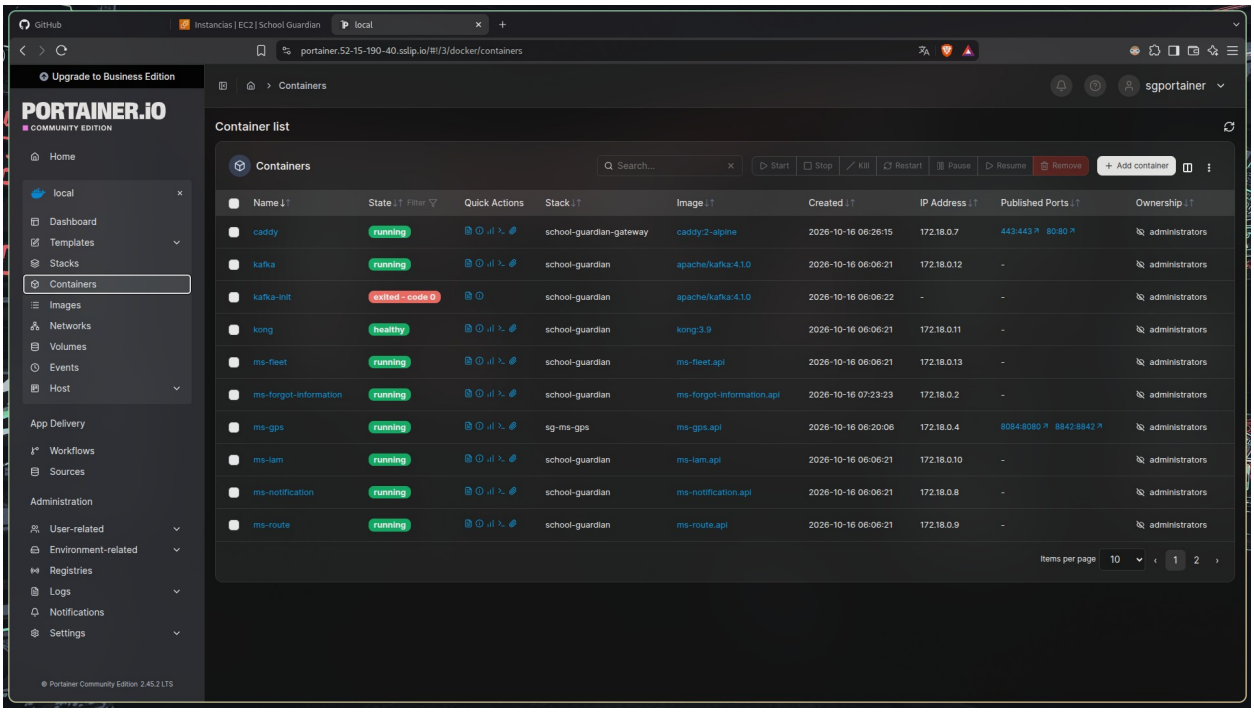
5.3.2 Monitoreo de consumo de CPU y memoria RAM en contenedores

```
docker stats --no-stream
free -h
```

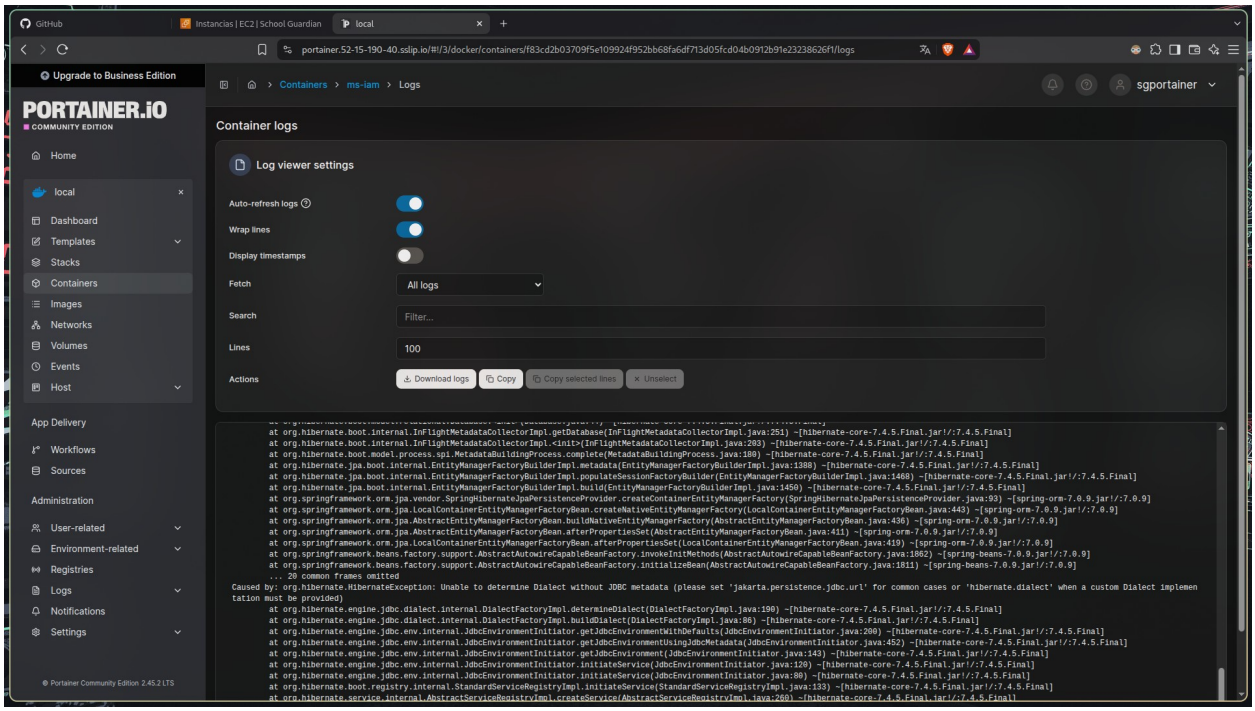
5.4 Registro de Evidencias Visuales (Screenshots)

Capturas de evidencia tomadas del entorno desplegado.

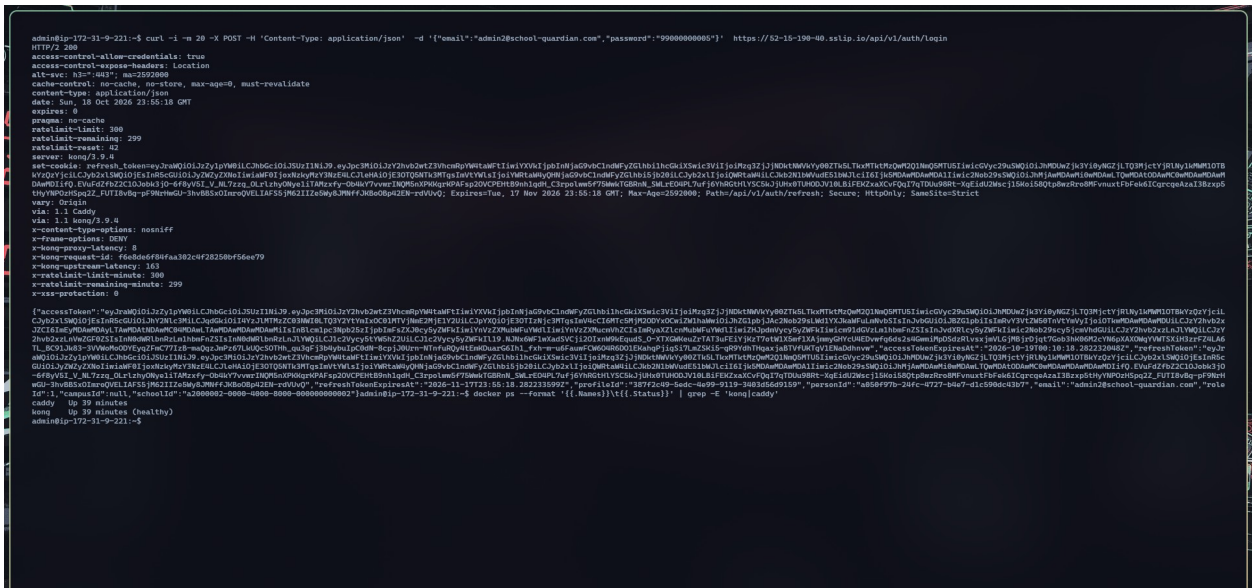
5.4.1 Captura del panel de Portainer con contenedores en estado Running



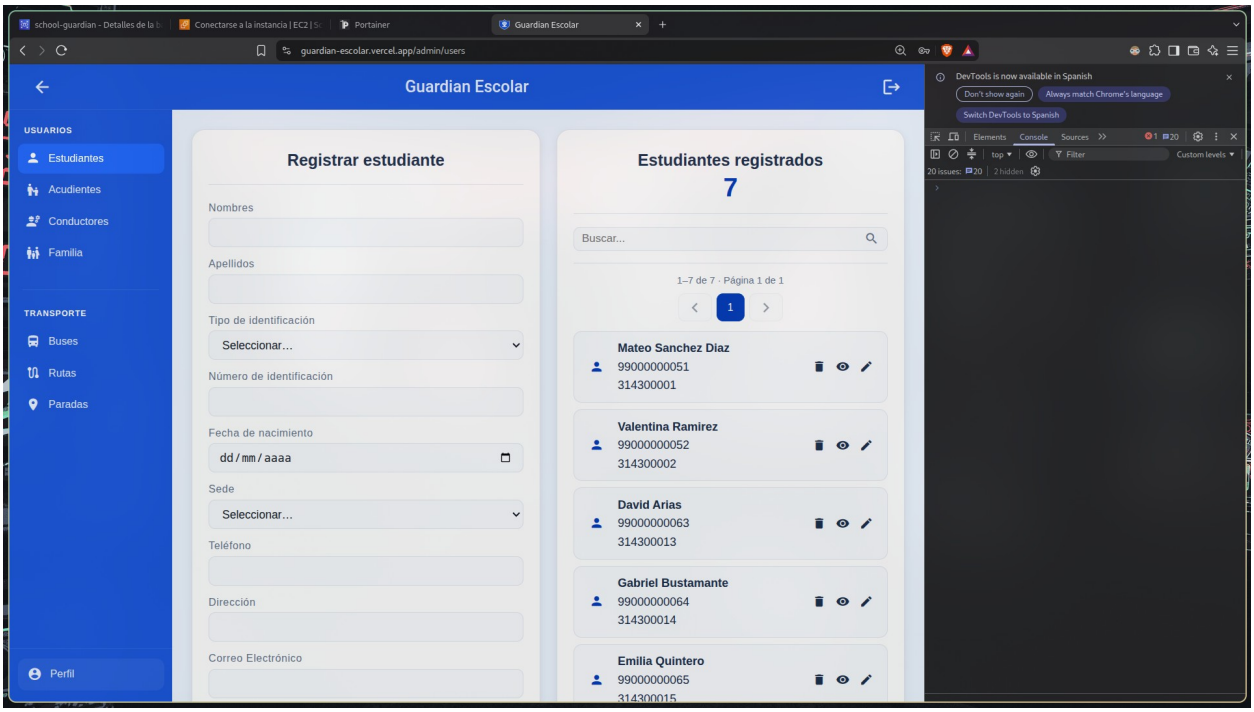
5.4.2 Captura de visor de logs de los servicios Backend



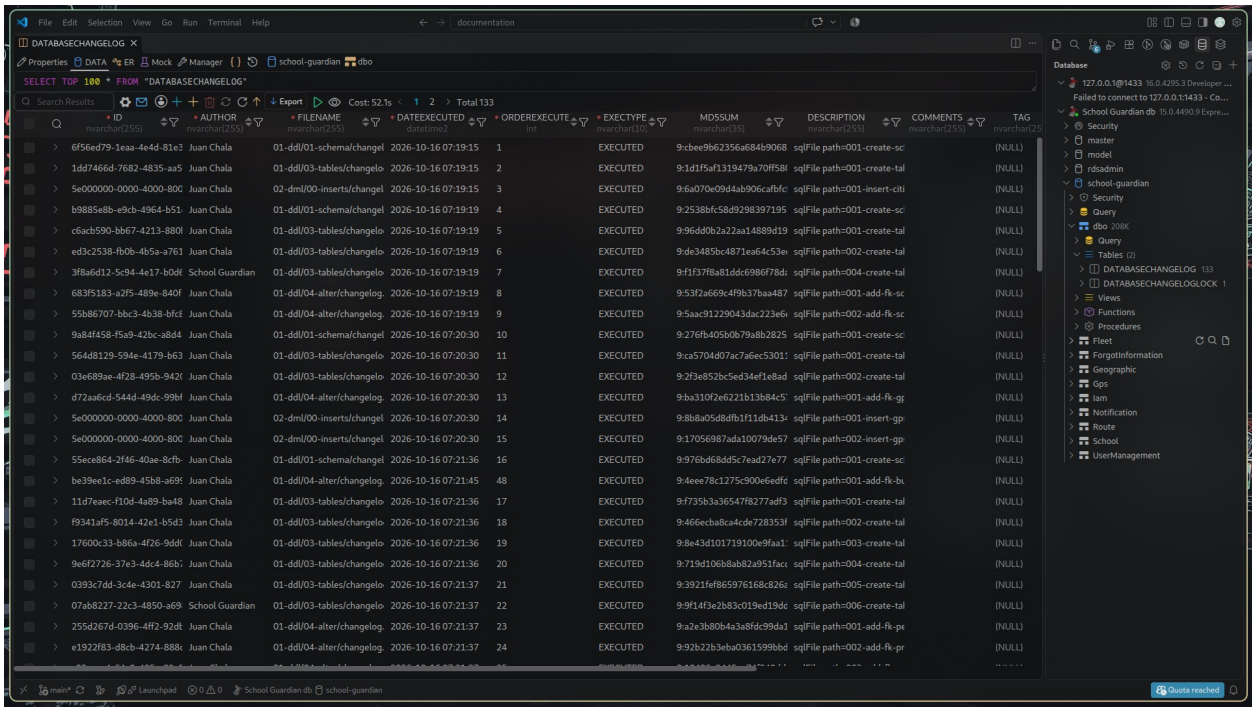
5.4.3 Captura de respuesta HTTP 200 de los Health Checks


admin@ip-172-31-9-221:~$ curl -i -m 20 -X POST -H 'Content-Type: application/json' -d '{"email":"admin2@school-guardian.com","password":"99000000005"}' https://52-15-198-40.sslip.io/api/v1/auth/login
HTTP/2 200
access-control-allow-credentials: true
access-control-expose-headers: Location
alt-svc: h3=":443"; ma=2592000
cache-control: no-cache, no-store, max-age=0, must-revalidate
content-type: application/json
date: Sun, 18 Oct 2026 23:55:18 GMT
expires: 0
pragma: no-cache
ratelimit-limit: 300
ratelimit-remaining: 299
ratelimit-reset: 42
server: kong/3.9.4
set-cookie: refresh_token=eyJraWQiOiJzZyIpYW0iLCJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJzY2hvb2wtZ3VhcmRpYWRtaWFtIiwiYXVkIjpbInNjaG9vbC1ndWFyZGlhbi1hcGkiXSwic3ViIjoiMzg3ZjJjNDktNWVkYy00ZTk5LTkxMTktMzQwMZQ1NmQ5MTU5IiwicGVyc29uSWQiOiJhMDUwZjk3Yi0yNGZjLTQ3MjctYjRlNy1kMWM1OTBkYzQzYjciLCJyb2xlSWQiOjEsInR5cGUiOiJyZWZyZXNoIiwiaWF0IjoxNzkyMzY3NzE4LCJleHAiOjE3OTIzNjc3MTgsImV4cCI6MTc5MjM2OTYxOCwiZWl1haWwiOiJhZGlpbi1Ac2Nob29sLWd1YXJkaWFuLmNvbSIsInJvbGUiOiJBZG1pbiIsImRvY3VtZW50TnVtYmVyIjoiOTkwMDAwMDAwMDAiLCJzY2hvb2xzIJCI6ImEyMDAwMDAtNDAwMC04MDAwLTAwMDAwMDAwMDAwMiIsInBlcm1pc3Npb25zIjpbImFsZXJcdmlld3MiLCJhY2hvb2xzLnJlYWQiLCJhY2hvb2xzLmNyZWF0ZSIsInNhbWRlbnRzLnJlYWQiLCJ1c2Vycy5tYW5hZ2UiLCJ1c2Vycy5jcmVhdGUiLCJhdHRlbmRhbmNlLnJlYWQiLCJhdHRlbmRhbmNlLnVwZGF0ZSIsInNhbWRlbnRzLnVwZGF0ZSIsInNhbWRlbnRzLnJlYWQ

5.4.4 Captura de interfaz web React/Angular con consola DevTools limpia



5.4.5 Captura de registro de migración exitosa en base de datos



---

6. Anexos

6.1 Datos de acceso y direcciones

Recurso	Valor
Dominio API (Caddy + Kong)	`https://<HOST>`
Dominio Portainer	`https://portainer.<HOST>`
IP pública (Elastic IP)	`<IP>`
Web (Vercel)	`https://guardian-escolar.vercel.app`
GPS TCP (VT03F)	`<IP>`
IMEI tracker	`355468590730586`
RDS Host	`<HOST_BASE_DATOS>:1433`
Instancia EC2	`i-0911016bb4a900105` (us-east-2)

6.2 Cuentas de prueba (entorno de testing)

El entorno de testing usa las cuentas semilla cargadas por Liquibase. Sus identificadores y hashes están definidos en las migraciones de `ms-user-management-db` y `ms-iam-db`; por política de seguridad este manual **no publica usuarios ni contraseñas**.

> **Antes de operar con datos reales:** restablecer la contraseña de todas las cuentas semilla y deshabilitar o eliminar las cuentas de demostración. No reutilizar estas cuentas en producción.

6.3 Acceso SSH

```
ssh -i <RUTA_LOCAL> admin@<IP>
```

6.4 Servicio GPS (ms-gps) — configuración del tracker VT03F

```
Servidor: <IP>
Puerto:   8842
Protocolo: TCP
```

Comando SMS de ejemplo (formato VT03F):

```
SERVER,1,<IP>,8842,0#
```

Verificación:

```
docker logs -f ms-gps      # esperar: "GPS conectado desde: /XX.XX.XX.XX:XXXX"
curl https://<HOST>/api/gps/devices/355468590730586/connected
# false → true al conectar el tracker
```

6.5 Frontend Web (Vercel) — variables de entorno

```
API_URL=https://<HOST>
API_FORGOT_INFORMATION_URL=https://<HOST>
```

6.6 App Móvil (React Native + Expo) — variables y compilación

Variables EAS (entorno `production`):

```
EXPO_PUBLIC_API_URL=https://<HOST>
EXPO_PUBLIC_FORGOT_INFORMATION_API_URL=https://<HOST>
EXPO_PUBLIC_GOOGLE_MAPS_API_KEY=***
# + variables de Notification API
```

Compilación Android (.APK) — perfil `production`:

```
cd dvlp-front/dvlp-movil/guardian-escolar
npx --yes eas-cli@latest build -p android --profile production --non-interactive
```

Build de referencia (EAS): **16 oct 2026, 2:39 PM** · cola 3 s · build 21 m 50 s · **total 21 m 58 s** · perfil `production`.

Compilación iOS (.IPA) — requiere cuenta Apple Developer e inicio de sesión interactivo:

```
npx --yes eas-cli@latest build -p ios --profile production
```

6.7 Troubleshooting

Problema	Causa	Solución
Web login no responde / todo vacío	IP antigua (instancia cambió de IP)	Apuntar a la Elastic IP `<IP>` en Vercel/EAS/móvil
Portainer "Origin invalid" al entrar	Portainer 2.27.7+ detrás de proxy	Flag `--trusted-origins https://portainer.<HOST>` en el compose
Portainer en bucle de reinicio	`--trusted-origins` sin esquema	Usar `https://host` (imagen LTS)
Web carga pero resto vacío	Kong `/user/*` mal mapeado + columnas BD faltantes	Ruta `/user/api` + `strip_path: true`, y columnas §4.2.2
`/schools` 500	`School.Phone` INT vs long; sin Lat/Long	`ALTER COLUMN Phone BIGINT` + columnas faltantes
GPS `/connected` global 404	El endpoint no existe	Usar `/devices/{imei}/connected`
Tracker no conecta	Apunta a IP antigua	Reconfigurar VT03F a `<IP>`
`ms-iam` caído tras reinicio	Falla de conexión a RDS en el arranque	`docker restart ms-iam`

6.8 Respaldos relevantes

`kong.yml.bak-pre-user-v1`, `kong.yml.bak-pre-notif`, `kong.yml.bak-pre-ip`, `kong.yml.bak-pre-eip`, `Caddyfile.gateway.bak-pre-ip`, `Caddyfile.gateway.bak-pre-eip`, `Caddyfile.gateway.bak-pre-portainer`.